



HTOC

HEALTHCARE THREAT OPERATIONS CENTER

PRISM Threat Assessment Scoring — ThreatAssessScoringV4

Version 4
April 2026

Table of Contents

Table of Contents

Table of Contents	2
1. Purpose	4
2. Scope	4
3. Prerequisites	4
3.1 Environment	4
3.2 Configuration File	5
3.3 Input Data Files	5
4. Key Configuration Parameters	6
5. Pipeline Overview	7
6. Step-by-Step Execution	8
Step 1 — Install and Import Dependencies	8
Step 2 — ThreatConnect Indicator Pull	8
Step 3 — Threat Actor Tags	9
Step 4 — First-Seen Date Merge	9
Step 5 — Incidents, Events, and Tags	9
Step 6 — OpDiv Observations and Partner Detection	9
Step 7 — API Enrichment (VirusTotal and Shodan)	10
Step 8 — Annual Observation Count	10
Step 9 — PRISM Rule Score Calculation	11
Step 10 — AI Layer (HistGradientBoostingRegressor)	13
Step 11 — Hybrid Score and Severity	14
Step 12 — Excel Export	14
Step 13 — Score History Utilities	15
7. Interpreting Results	16
8. Troubleshooting	16
9. Maintenance Notes	17
10. How to Run the Notebook	18
10.1 Open the Notebook	18
10.2 Verify Prerequisites	18
10.3 Run All Cells	18
10.4 Monitor Progress	19

10.5 Review Output	19
10.6 Typical Run Time	20
10.7 Scheduling Considerations	20
11. Appendix - Standalone Python Script	21
12. Related Documents	21

1. Purpose

This SOP describes how to operate the PRISM (Prioritized Risk Indicator Severity Model) pipeline. The notebook connects to ThreatConnect, enriches threat indicators with local and third-party data, computes a rule-based risk score blended with a machine learning layer, and exports prioritized results to Excel for analyst consumption.

-

2. Scope

This procedure applies to HTOC analysts and data engineers who run, maintain, or interpret threat indicator scoring outputs. It covers end-to-end execution of ThreatAssessScoringV4.ipynb, from environment setup through Excel output review.

-

3. Prerequisites

3.1 Environment

Requirement	Notes
Python 3.13	Verified against `cp313` wheel artifacts
Jupyter (VS Code or JupyterLab)	Notebook must be run interactively
scikit-learn	Installed automatically by first cell (`pip install scikit-learn`)
numpy, pandas, openpyxl	Required; typically already installed

Requirement	Notes
Network access to ThreatConnect API	SSL verification is disabled in the notebook
Access to `Z:\HTOC\Data_Analytics\`	All input CSVs and Excel output live here
ThreatConnect SDK	Located at `Z:\HTOC\Data_Analytics\threatconnect`

Example dependency install command:

```
pip install scikit-learn
```

3.2 Configuration File

The API connection reads from:

```
...\ThreatConnect-api-pull\utils\config.json
```

Confirm this file exists and contains valid API credentials before each run. Contact the HTOC admin if credentials have rotated.

3.3 Input Data Files

All CSV inputs must be up to date prior to execution:

File	Location	Purpose
`htoc_observed_indicator_tags.csv`	`Z:\HTOC\Data_Analytics\Data\Observed_Tags\`	Threat actor tag associations
`htoc_observed_indicators.csv`	`Z:\HTOC\Data_Analytics\Data\Observed_Indicators\`	First-seen timestamps

File	Location	Purpose
`htoc_opdiv_obs_d{YYYYMMDD}.csv`	`Z:\HTOC\Data_Analytics\Data\OpDiv_Observations\`	OpDiv observation records (daily files)

Ensure daily OpDiv files cover at least the last 30 days (60 days recommended for partner detection accuracy).

-

4. Key Configuration Parameters

These variables are defined near the top of the notebook and control pipeline behavior. Verify or adjust them before each run.

Parameter	Default	Description
`QUERY_LOOKBACK_DAYS`	30	Days back to pull indicators from ThreatConnect (by `lastObserved`)
`RESULT_PAGE_SIZE`	500	Indicators per API page
`FIRSTSEEN_LOOKBACK_DAYS`	13	New indicator window for first-seen bonus
`OPDIV_LOOKBACK_DAYS`	30	Days of OpDiv files to load for partner detection

Parameter	Default	Description
`FIRST_OBS_MAX_DAYS`	14	Days over which first-seen linear decay is applied
`VT_MAX`	94	Maximum possible VirusTotal detection count
`VT_EFFECTIVE_MAX`	40	VT detections are clipped at this value for scoring
`OBS_PENALTY_STRENGTH`	(configured)	Strength of observation-frequency penalty

-

5. Pipeline Overview

The notebook executes in the following logical stages. Run cells in order from top to bottom unless troubleshooting a specific section.

```
[1] Install / Import Dependencies
    ↓
[2] ThreatConnect Connection & Indicator Pull
    ↓
[3] Enrich: Threat Actor Tags, First-Seen, Incidents/Events
    ↓
[4] Enrich: OpDiv Observations & Partner Detection
    ↓
[5] API Enrichment (VirusTotal, Shodan)
    ↓
[6] Annual Observation Count (365-day reload)
    ↓
[7] PRISM Rule Score Calculation
    ↓
[8] AI Layer (HistGradientBoostingRegressor)
    ↓
[9] Hybrid Score & Severity Assignment
    ↓
[10] Excel Export & History Append
```

-

6. Step-by-Step Execution

Step 1 — Install and Import Dependencies

Run the first cell. The `pip install scikit-learn` line installs scikit-learn and dependencies (scipy, joblib, threadpoolctl) if not already present. Subsequent import cells bring in pandas, numpy, openpyxl, and the ThreatConnect SDK.

Verify: No import errors. The ThreatConnect SDK path (<Z:\HTOC\Data Analytics\threatconnect>) must be accessible.

-

Step 2 — ThreatConnect Indicator Pull

The notebook queries ThreatConnect using TQL across multiple owners and indicator types for indicators observed within `QUERY_LOOKBACK_DAYS`. Results are paginated at `RESULT_PAGE_SIZE` per page. After retrieval, only indicators where `ownerName == 'HTOC Org'` are retained.

Fields retrieved include: `summary`, `type`, `rating`, `confidence`, `threatAssessScore`, `calScore`, `lastObserved`, `firstSeen`, `falsePositives`, `tags`, `associatedGroups`, and `description`.

Verify: A non-empty dataframe is printed. If zero results are returned, check API credentials and TQL query window.

-

Step 3 — Threat Actor Tags

Loads `htoc_observed_indicator_tags.csv`, filters for rows where `threat_category == 'THREAT ACTOR'`, and merges associated threat actor names into the indicators dataframe under the `threat_actor` column.

-

Step 4 — First-Seen Date Merge

Loads `htoc_observed_indicators.csv` and merges `firstseen_date` onto indicators whose first-seen timestamp falls within the last `FIRSTSEEN_LOOKBACK_DAYS` days. This enables the first-seen scoring bonus.

-

Step 5 — Incidents, Events, and Tags

Incidents/Events: Extracted from the `associatedGroups.data` field (type incident or event) and from `INC...-pattern` references in the description field. Stored as the [incidents/events](#) column.

Tags: Tag lists are exploded to one row per tag. Botnet-related tags are identified using the `BOTNET_TAGS_OF_INTEREST` list and stored in the `Botnet` column.

-

Step 6 — OpDiv Observations and Partner Detection

Loads daily OpDiv CSV files for the past `OPDIV_LOOKBACK_DAYS` days. Partners are identified from two sources:

OpDiv-based: Organizations that observed the indicator within 60 days of the cutoff date.

Tag-based: Tags matching the * Splunk API prefix pattern or standalone partner tag names in KNOWN_PARTNERS.

Results are stored as partners (list) and partner_count (integer).

-

Step 7 — API Enrichment (VirusTotal and Shodan)

The notebook sends parallel POST requests to [/v3/indicators/{id|value}/enrich](#) for eligible indicator types. Enrichment results are flattened into enrich_* columns. Key enrichment fields include:

enrich_vtMaliciousCount — VirusTotal malicious detection count

enrich_tags — enrichment-derived tags (used for TOR detection)

Note: Missing VirusTotal data is treated as **neutral** (zero) for scoring purposes, not as a penalty.

-

Step 8 — Annual Observation Count

Reloads OpDiv files for the past 365 days. For each indicator, counts the number of unique observation dates (obs_count). This value drives the observation continuity penalty.

-

Step 9 — PRISM Rule Score Calculation

The rule-based score is computed using the following additive components and modifiers:

9.1 Additive Score Components

Component	Weight	Notes
VirusTotal Malicious	7.50	<code>`vt_effective ** 0.75 * MALICIOUS_WEIGHT`</code> ; VT clipped at <code>`VT_EFFECTIVE_MAX = 40`</code>
TOR Activity	9.00	Doubled if VT present and count ≥ 10
Threat Actor	10.00	Binary (1 if threat actor tag present)
Incidents/Events	8.00	Binary bonus if associated incidents or events exist
CAL Score	2.75	<code>`calScore / 1000 * weight`</code>
Sources (HTOC)	2.80	<code>`log1p(sources_count - 1) * weight`</code>
Partners	2.10	<code>`log1p(partner_count - 1) * weight`</code>
TC Threat Score	0.75	<code>`threatAssessScore / 1000 * weight`</code>
Continuity	0.90	Mapped by indicator type; file/hash types receive a 900-point baseline
TC Rating	0.01	Raw rating value $\times$ weight
TC Confidence	0.025	<code>`sqrt(confidence) * weight`</code>
Observation Count	0.02	<code>`obs_count * weight`</code>
First-Seen Bonus	2.00	Linear decay from <code>`FIRSTSEEN_LOOKBACK_DAYS`</code> to <code>`FIRST_OBS_MAX_DAYS`</code>
Stacked Context Bonus	—	+25 if ≥ 4 signals; +15 if 3 signals; +7 if 2 signals

Stacked signals counted: threat actor present, TOR active, incidents/events present, sources ≥ 2 , partners ≥ 2 .

9.2 Multipliers and Penalties (applied after raw sum)

Modifier	Factor	Condition
Observation frequency penalty	0.50–1.00	Scales with `obs_count / 365`; penalizes very commonly observed indicators
Data quality	0.85 minimum	Applied when `type`, `rating`, or `confidence` are incomplete
False positives	$\times 0.90$	Applied if `falsePositives > 0`
Scanner tag	$\times 0.95$	Applied if scanner tag detected (except file hashes)
Botnet actions	$\times 0.40$	Applied if botnet action tags match `BOTNET_ACTIONS` list (non-file types)

9.3 Normalization

Raw score is divided by a theoretical BASE_CAP (adjusted for file types), then:

```
PRISM_Score = clip(raw_ratio * 1000, 0, 1000) * 1.40, capped at 1000
```

The $\times 1.40$ calibration factor adjusts for real-world score distribution.

9.4 Post-Normalization Caps and Floors

Condition	Action
$VT \leq 3$	Cap score at 199 (Low), unless TOR boost exception applies
$VT \geq 13$	Floor score at 499 (Medium)
Botnet action tag (non-file)	Cap score at 199, unless TOR exception
File / hash indicator types	Force score to 1000 (Critical)

-

Step 10 — AI Layer (HistGradientBoostingRegressor)

A gradient boosting regression model is trained on the current run's data to learn the rule-score pattern:

Features: VT count, obs_count, rating, confidence, calScore, threatAssessScore, TOR flag, incidents flag, sources count, partner count, threat actor score, first-seen days, stacked bonus, botnet flag, false positive flag, scanner multiplier, data quality multiplier.

Target: PRISM_Score from the rule layer.

Split: 80% train / 20% test, random_state=42.

Hyperparameters: max_depth=6, learning_rate=0.05, max_iter=300.

MAE and R^2 are printed after training. Expected R^2 should be > 0.90 on a healthy run; investigate if substantially lower.

-

Step 11 — Hybrid Score and Severity

The final score blends rule and AI outputs:

$$\text{PRISM_Score_Final} = 0.70 * \text{PRISM_Score} + 0.30 * \text{PRISM_Score_AI}$$

File/hash types bypass the AI layer and use the rule score directly.

Severity levels are assigned using the following bins:

Score Range	Severity
0 – 199	Low
200 – 499	Medium
500 – 799	High
800 – 1000	Critical

Output columns include: PRISM Score, PRISM Score (AI), PRISM Score (Final), Severity (Final), Explanation, and AI_Adjustment.

-

Step 12 — Excel Export

Results are written to:

Z:\HTOC\Data_Analytics\Data\Threat Assessment Scores\
Prioritized_Risk_Indicator_Severity_Model_Scores.xlsx

The workbook contains three sheets:

Sheet	Contents
PRISM Scores	Current run results with severity row coloring
Score Comparison	Side-by-side PRISM vs. ThreatConnect native score
Complete History	Append-only log of all scoring runs (deduplicated by indicator + date)

Rows are color-coded by severity for quick triage. The Complete History sheet is never overwritten; each run appends new records.

-

Step 13 — Score History Utilities

Two helper functions are available for post-run analysis:

Function	Usage
<code>`get_indicator_score_history(indicator)`</code>	Returns all historical scores for a specific indicator value
<code>`get_score_changes_since(days_ago=7)`</code>	Returns indicators whose scores have changed in the last N days

Run these in a new cell at the bottom of the notebook as needed.

-

7. Interpreting Results

Severity	PRISM Score	Recommended Action
Critical	800–1000	Immediate triage; escalate to relevant OpDiv teams
High	500–799	Prioritize for analyst review within 24 hours
Medium	200–499	Review during normal operations cycle
Low	0–199	Monitor; review if score increases in subsequent runs

Review the `AI_Adjustment` column to identify indicators where the AI layer significantly diverges from the rule score. Large divergence ($> \pm 100$) warrants manual inspection of the underlying features.

-

8. Troubleshooting

Symptom	Likely Cause	Resolution
Zero indicators returned	API credentials expired or TQL window too narrow	Check <code>config.json</code> ; increase <code>QUERY_LOOKBACK_DAYS</code>
<code>'KeyError'</code> on CSV load	Missing or renamed input file	Verify all input files exist at expected paths
<code>'enrich_domains'</code> column missing on <code>'exploded'</code>	Expected; enrichment not available for that indicator type	Non-blocking; domain count cell handles gracefully

Symptom	Likely Cause	Resolution
$R^2 < 0.85$ on AI layer	Very small dataset or unusual score distribution	Inspect `PRISM_Score` distribution; may need manual review
`VirusTotal Malicious Score` missing from Excel export	Column rename mismatch between scoring and export cells	Verify column naming in export `columns_to_save` list
OpDiv files not found	Daily files not yet generated for target dates	Confirm OpDiv feed is current; reduce `OPDIV_LOOKBACK_DAYS` if needed
SSL errors on ThreatConnect	Network/proxy changes	SSL verify is disabled in the notebook; contact network admin if errors persist

-

9. Maintenance Notes

Version history: This is Version 4 of the scoring notebook. Previous versions are archived in the same folder.

Weight tuning: All scoring weights are defined in the `weights` dictionary near the top of the scoring cell. Adjustments require analyst-level review and should be documented with a rationale comment in the notebook.

Model retraining: The AI layer is retrained on each run against the current rule scores. No persistent model file is saved. This is by design — the model adapts to the current indicator population.

Score caps/floors: VT-based caps ($\leq 3 \rightarrow$ Low cap; $\geq 13 \rightarrow$ Medium floor) are intentional policy decisions. Changes require HTOC leadership approval.

Botnet penalty: The $\times 0.40$ botnet multiplier applies only to `BOTNET_ACTIONS`-matched tags, not all botnet tags. SQL Injection and Cross-Site Scripting are in the tag list but excluded from `BOTNET_ACTIONS`.

-

10. How to Run the Notebook

10.1 Open the Notebook

Open **Cursor** (or VS Code / JupyterLab).

Navigate to H:\HTOC\notebooks\ThreatAssessment_Scoring\.

Open ThreatAssessScoringV4.ipynb.

10.2 Verify Prerequisites

Before executing any cells, confirm the following:

- [] You have network connectivity to the ThreatConnect API.
- [] Z:\HTOC\Data_Analytics is accessible and mapped.
- [] config.json at <...\ThreatConnect-api-pull\utils\config.json> exists and contains current credentials.
- [] Daily OpDiv CSV files in Z:\HTOC\Data_Analytics\Data\OpDiv_Observations are current (at minimum, files for the past 30 days must be present).
- [] The observed tags and observed indicators CSVs have been refreshed for today's run.

10.3 Run All Cells

Use **Run All** to execute the notebook from top to bottom:

Cursor / VS Code: Click the ►► Run All button in the notebook toolbar, or open the Command Palette (Ctrl+Shift+P) and select **Notebook: Run All Cells**.

JupyterLab: Kernel → Restart Kernel and Run All Cells... → confirm restart.

> **Important:** Always run cells in sequential order. Running individual cells out of order will produce incorrect results because each stage depends on the dataframe state built by the previous stage.

10.4 Monitor Progress

The notebook prints status output after each major stage. Watch for:

Stage	Expected Output
Indicator pull	A dataframe preview showing pulled indicators with `ownerName`, `type`, `summary`, etc.
Enrichment	A progress indicator for parallel API requests; `enrich_*` columns appear in the dataframe
PRISM scoring	A scored dataframe with `PRISM_Score`, `Severity`, and `Explanation` columns
AI layer	Printed MAE and R² values (R ² should be > 0.90)
Excel export	A confirmation message with the output file path

If a cell raises an exception, do not continue running subsequent cells. Refer to the **Troubleshooting** section (Section 8) to resolve the issue before restarting.

10.5 Review Output

Once execution completes:

Open the Excel file at:

```
Z:\HTOC\Data_Analytics\Data\Threat Assessment Scores\  
Prioritized_Risk_Indicator_Severity_Model_Scores.xlsx
```

Review the **PRISM Scores** sheet for today's run. Rows are color-coded by severity.

Use the **Score Comparison** sheet to compare PRISM scores against ThreatConnect native scores.

Optionally run the helper functions in a new cell at the bottom of the notebook to query score history:

```
get_indicator_score_history("203.0.113.45")    # example IP  
get_score_changes_since(days_ago=7)
```

10.6 Typical Run Time

Phase	Approximate Duration
Dependency install (first run only)	2–5 minutes
Indicator pull from ThreatConnect	2–10 minutes (varies by result count)
API enrichment (VT / Shodan)	5–15 minutes (parallel, varies by indicator count)
Scoring + AI training	< 1 minute
Excel export	< 30 seconds
Total (after first run)	**~10–30 minutes**

10.7 Scheduling Considerations

This notebook is intended to be run **manually** on an as-needed or recurring basis (e.g., daily or weekly). If automation is desired, export the notebook to a .py script and schedule it via Windows Task Scheduler or a workflow tool. Ensure the scheduled account has access to both the network drive ([Z:\](#)) and the ThreatConnect API.

-

11. Appendix - Standalone Python Script

The complete standalone script extracted from the notebook is attached at:

H:\HTOC\documentation\SOP\Appendix_Scripts\ThreatAssessScoringV4_standalone.py

Run with:

```
&"C:\Program Files\Python313\python.exe" "H:\HTOC\documentation\SOP\Appendix  
Scripts\ThreatAssessScoringV4_standalone.py"
```

-

12. Related Documents

ThreatConnect API documentation

HTOC OpDiv Observation Feed runbook

Z:\HTOC\Data_Analytics\Data\Threat_Assessment_Scores (output location)

Previous notebook versions: ThreatAssessScoringV1.ipynb through ThreatAssessScoringV3.ipynb